

# Aula 27 - Spring WebFlux + R2DBC com MySQL

Duração: 4 horas (1h teórica + 3h prática)

## Parte Teórica (1 hora)

### 1. R2DBC vs JDBC (20 min)

| Característica | R2DBC                                     | JDBC                 |
|----------------|-------------------------------------------|----------------------|
| Modelo         | Não-blocking (Reativo)                    | Blocking             |
| Conexões       | Mais eficientes (menos threads)           | 1 thread por conexão |
| Suporte a SQL  | MySQL, PostgreSQL, H2                     | Todos os bancos      |
| Transações     | Suportado ( <code>@Transactional</code> ) | Suportado            |

### 2. Configuração do MySQL Reativo (20 min)

#### Dependências Maven

xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-r2dbc</artifactId>
</dependency>
<dependency>
  <groupId>dev.miku</groupId>
  <artifactId>r2dbc-mysql</artifactId>
</dependency>
```

#### application.yml

yaml

```
spring:
  r2dbc:
    url: r2dbc:mysql://localhost:3306/reactive_db
    username: user
    password: senha
```

### 3. Fluxo de Trabalho (20 min)

1. Entidade → Repository → Controller (igual ao Spring Data JPA, mas reativo).
2. Operações Assíncronas:
  - o Tudo retorna `Mono` ou `Flux`.
  - o Nada é bloqueante (nem mesmo o banco de dados).

## Parte Prática (3 horas)

### Atividade: API de Pedidos Reativa

#### Passo 1: Modelagem (30 min)

##### 1. Entidade `Order` :

```
java

@Data @NoArgsConstructor @AllArgsConstructor
public class Order {
    @Id
    private Long id;
    private String customerName;
    private LocalDateTime orderDate;
    private Double totalAmount;
}
```

##### 2. Repository:

```
java

public interface OrderRepository extends ReactiveCrudRepository<Order, Long> {
    Flux<Order> findByOrderDateAfter(LocalDateTime date);
}
```

#### Passo 2: Configuração do MySQL (30 min)

##### 1. Criar Banco de Dados:

```
sql

CREATE DATABASE reactive_db;
CREATE TABLE orders (id BIGINT AUTO_INCREMENT PRIMARY KEY, customer_name VARCHAR(255), order_date DATETIME, total_amount DOUBLE);
```

##### 2. Popular Dados Iniciais:

java

```
@Bean
public CommandLineRunner initData(OrderRepository repo) {
    return args -> {
        repo.saveAll(List.of(
            new Order(null, "Cliente A", LocalDateTime.now(), 150.0),
            new Order(null, "Cliente B", LocalDateTime.now().minusDays(1), 200.0)
        )).subscribe(); // Subscribe para executar a operação
    };
}
```

## Passo 3: Controller Reativo (1h)

### 1. Endpoints CRUD:

java

```
@RestController
@RequestMapping("/orders")
@RequiredArgsConstructor
public class OrderController {
    private final OrderRepository repo;

    @GetMapping
    public Flux<Order> getAll() {
        return repo.findAll();
    }

    @PostMapping
    public Mono<Order> create(@RequestBody Order order) {
        return repo.save(order);
    }
}
```

### 2. Endpoint com Filtro:

java

```
@GetMapping("/recent")
public Flux<Order> getRecentOrders() {
    return repo.findByOrderDateAfter(LocalDateTime.now().minusDays(7));
}
```

## Passo 4: Testes (1h)

### 1. Teste de Integração:

java

```

@SpringBootTest
class OrderControllerTest {
    @Autowired
    private WebTestClient webClient;

    @Test
    void shouldReturnOrders() {
        webClient.get().uri("/orders")
            .exchange()
            .expectStatus().isOk()
            .expectBodyList(Order.class).hasSize(2);
    }
}

```

## 2. Teste com Postman:

- Verificar resposta do endpoint `/orders/recent` (deve retornar JSON stream).

## Tarefa para Casa

### 1. Relacionamentos Reativos:

- Implementar `Order` → `OrderItem` (one-to-many) com R2DBC.

### 2. Transações:

- Criar um serviço que use `@Transactional` para salvar pedido + itens atomicamente.

## Próxima Aula

- Tópico: Segurança em WebFlux (Spring Security Reactive).
- Atividade prática: Autenticação JWT em APIs reativas.

## Resultado Esperado

- API reativa com:
  - Operações CRUD não-blocking no MySQL.
  - Endpoints retornando `Mono` / `Flux`.
  - Testes automatizados com `WebTestClient`.

- ✓ MySQL configurado para R2DBC
- ✓ Controller 100% reativo
- ✓ Pronto para segurança reativa

<https://spring.io/images/r2dbc-sequence-9c1b5e5c5a5e8e5e5e5e5e5e5e5e5e5e.png> (*Fluxo R2DBC no Spring*) 🚀